Market Price Forecasting for Ugandan Staple Crops

A Multi-Model Time-Series Approach Using WFP Market Data and Climate Covariates

Jamugisa Peter Paul (B35503 – S25M25/008)
ILEGO Cynthia Cecilia (B35502 – S25M25/007)
AYEBARE Moses (B35498 – S25M25/003)
Apilli Barbra Ebal (B35496 – S25M25/001)

Department of Computer Science • Uganda Christian University
MSc Computer Science – AI & Machine Learning

April 2025

Presentation Outline

Presenter 1 – Jamugisa Peter Paul

§1 Introduction & Problem Statement

§2 The Data Pipeline

Presenter 2 – ILEGO Cynthia Cecilia

§3 Feature Engineering

§4 Model 1: ARIMA Baseline

Presenter 3 – AYEBARE Moses

§5 Model 2: Facebook Prophet

§6 Model 3: LSTM Neural Network

Presenter 4 – Apilli Barbra Ebal

§7 Results & Model Comparison

§8 Live Dashboard Demo

§9 Challenges & Conclusions

The Problem: Food Insecurity through Price Volatility

Presenter: Jamugisa Peter Paul *[0:00 – 2:30]*

- Agriculture = **24% of Uganda's GDP**
- Employs **70%+** of the workforce
- Staple crops: **Maize, Beans, Cassava**
- Prices fluctuate **30–50%** between seasons
- Existing WFP/FEWS NET systems are **retrospective** – they tell us what prices *are*, not what they *will be*

Our Solution

Build a machine learning system that forecasts crop prices **4–8 weeks ahead** at district level

Impact for Stakeholders

- ✓ **Farmers** optimise selling timing
- ✓ **NGOs** pre-position food aid
- ✓ **Policymakers** plan interventions

Research Question

Which model – ARIMA, Prophet, or LSTM – forecasts best when rainfall is a predictor?

Research Objectives

Presenter: Jamugisa Peter Paul

1. **Collect & Merge** WFP prices + CHIRPS rainfall + World Bank commodity indices
2. **Engineer Features:** lags, rolling stats, seasonal encoding, climate covariates
3. **Implement 3 Models:** ARIMA, Facebook Prophet, LSTM
4. **Evaluate & Compare** using MAE, RMSE, MAPE (80/20 chronological split)
5. **Deploy** best model in an interactive Streamlit dashboard

Scope

3 Crops	**3 Districts**	**14 Years**
Maize, Beans, Cassava	Kampala, Gulu, Mbale	2010–2024

Three Heterogeneous Data Sources

Presenter: Jamugisa Peter Paul *[2:30 – 5:00]*

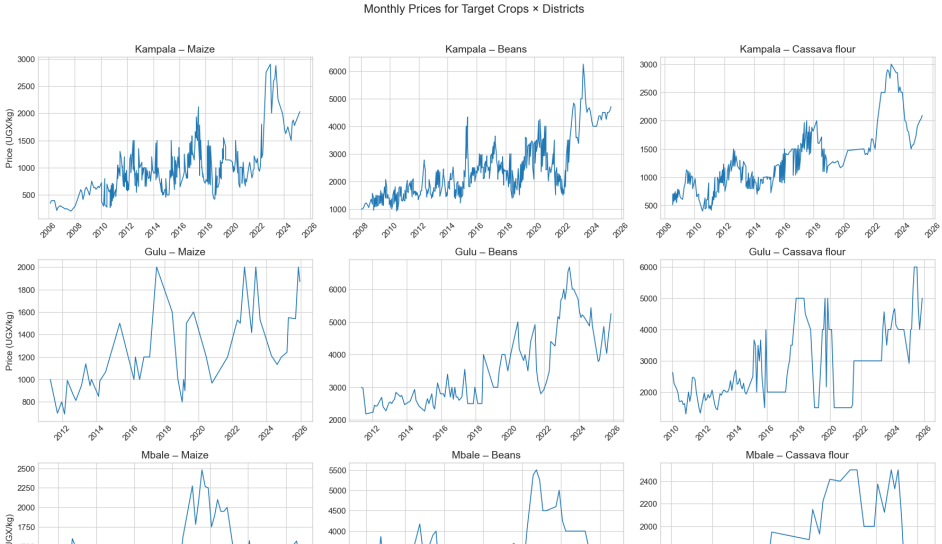
Dataset	Source	Content	Role
WFP Food Prices	Humanitarian Data Exchange	Market prices (UGX) across districts	**Target variable**
CHIRPS Rainfall	UC Santa Barbara satellite	Monthly rainfall (mm) by region	**Climate covariate**
World Bank Commodities	World Bank	Global crude oil & maize indices	**Global proxy**

Why These Three?

- **Rainfall:** Uganda's agriculture is rain-fed – drought today = high prices in 4 months
- **Crude Oil:** Drives transport costs on Uganda's unpaved roads
- **Global Maize:** International supply-demand spills into local markets via trade

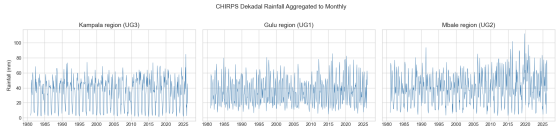
Price Trends Across All Crop-District Pairs

Presenter: Jamugisa Peter Paul *[5:00 – 5:45]*

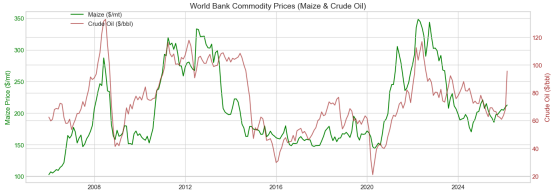


Rainfall Patterns & World Bank Commodity Trends

Presenter: Jamugisa Peter Paul *[5:45 – 7:00]*



CHIRPS rainfall by region – clear bimodal wet season pattern



Global crude oil & maize prices – transport cost proxies

Transforming Raw Data into AI-Ready Features

Presenter: ILEGO Cynthia Cecilia *[7:00 – 10:00]*

1. Price Lag Features

$$\text{price_lag}_k(t) = \text{price}(t-k)$$

$k \in \{1, 2, 4, 8\}$ – captures momentum

2. Rolling Statistics

$$\text{roll_mean}_w = \frac{1}{w}\sum_{i=0}^{w-1} \text{price}(t-i)$$

Windows: 4 & 8 months (mean + std dev)

3. Sinusoidal Seasonal Encoding

$$\text{month_sin} = \sin\left(\frac{2\pi m}{12}\right)$$

$$\text{month_cos} = \cos\left(\frac{2\pi m}{12}\right)$$

Preserves cyclical continuity (Dec $\leftrightarrow$ Jan)

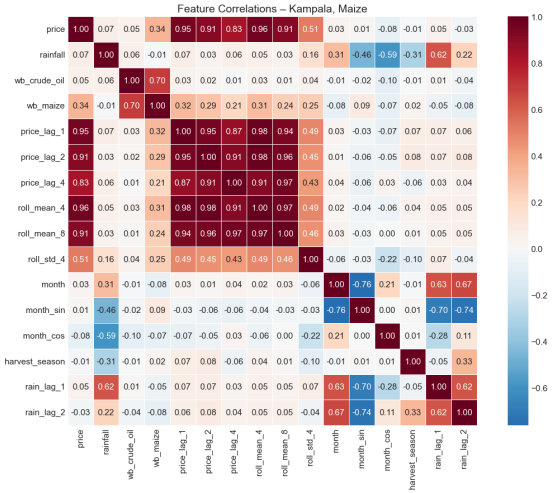
4. Climate & Global Lags

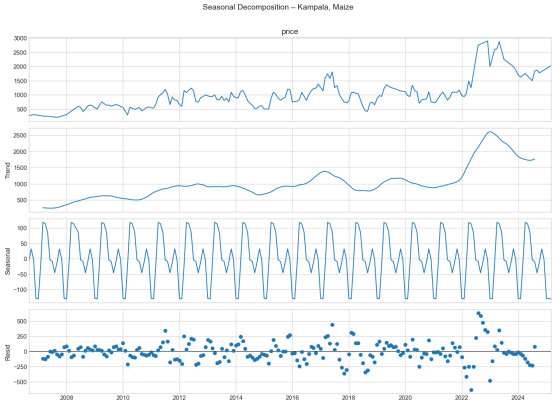
$$\text{rain_lag}_k(t) = \text{rainfall}(t-k)$$

$k \in \{1, 4, 8\}$ + WB oil & maize indices

Feature Analysis: Correlation & Seasonality

Presenter: ILEGO Cynthia Cecilia





Seasonal decomposition – clear yearly pattern validates seasonal encoding

ARIMA: Classical Statistical Baseline

Presenter: ILEGO Cynthia Cecilia *[10:00 – 12:30]*

ARIMA(p, d, q) equation:

$$y'_t = c + \sum_{i=1}^{p} \phi_i y'_{t-i} + \sum_{j=1}^{q} \theta_j \varepsilon_{t-j} + \varepsilon_t$$

- **AR(p):** Linear combination of p past values
- **I(d):** Differencing for stationarity
- **MA(q):** Past forecast errors for self-correction

Order Selection: AIC grid search

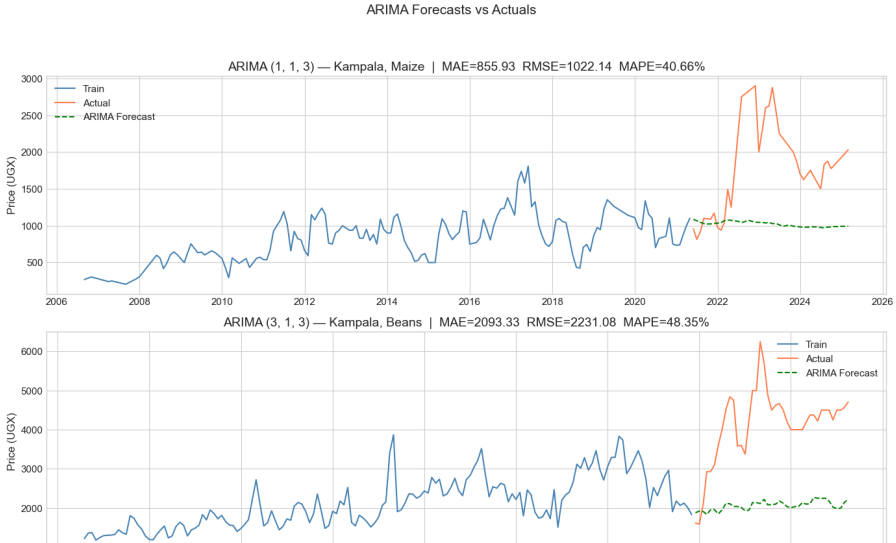
$$\text{AIC} = 2k - 2\ln(\hat{L})$$

$p \in \{0..3\}$, $d \in \{0, 1\}$, $q \in \{0..3\} = 32$ candidates

```python
# ARIMA AIC Grid Search
from statsmodels.tsa.statespace
    .sarimax import SARIMAX

best_aic = float('inf')
for p in range(4):
  for d in range(2):
    for q in range(4):
      model = SARIMAX(
        train['price'],
        order=(p, d, q))
      result = model.fit(
        disp=False)
      if result.aic < best_aic:
        best_aic = result.aic
        best_order = (p, d, q)
```

ARIMA Forecast Results

Presenter: ILEGO Cynthia Cecilia *[12:30 – 14:00]*



Prophet: Additive Decomposition with Rainfall

Presenter: AYEBARE Moses *[14:00 – 16:00]*

Prophet Decomposition:

$$y(t) = g(t) + s(t) + \beta_{\text{rain}} \cdot \text{rainfall}(t) + \varepsilon_t$$

- $g(t)$ = **Piecewise linear trend** with automatic changepoint detection
- $s(t)$ = **Fourier seasonal component**:

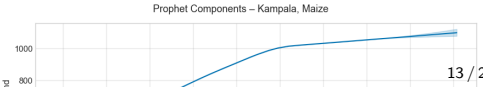
$$s(t) = \sum_{n=1}^{N} \left(a_n \cos \frac{2\pi nt}{P} + b_n \sin \frac{2\pi nt}{P} \right)$$

- β_{rain} = Learned rainfall coefficient

```python
# Prophet with Rainfall
from prophet import Prophet

m = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=False,
    daily_seasonality=False)

# Add rainfall as regressor
m.add_regressor('rainfall')
m.fit(train_df)

future = m.make_future_dataframe(
    periods=len(test))
forecast = m.predict(future)
```

Prophet: Results & Component Decomposition

Presenter: AYEBARE Moses *[16:00 – 17:30]*



LSTM: Deep Learning Architecture

Presenter: AYEBARE Moses *[17:30 – 20:00]*

LSTM Gating Mechanism:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad \text{Forget}$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad \text{Input}$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad \text{Output}$$

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{Cell State}$$

- Cell state = **"memory highway"** – carries information across time steps
- Solves **vanishing gradient problem** of standard RNNs

Our Architecture

Layer 1	LSTM (64 units)
Dropout	20%
Layer 2	LSTM (32 units)
Dropout	20%
Output	Dense (1 unit)
Window	6 months
Features	9 inputs
Optimiser	Adam
Loss	MSE
Epochs	100 (early stop)

LSTM: Implementation & Training Code

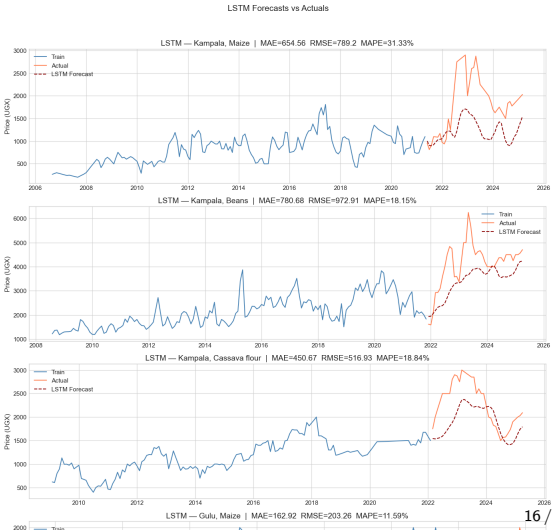
Presenter: AYEBARE Moses *[20:00 – 21:00]*

```python
# LSTM Architecture (TensorFlow/Keras)
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    LSTM, Dense, Dropout)

model = Sequential([
    LSTM(64, return_sequences=True,
         input_shape=(6, 9)),
    Dropout(0.2),
    LSTM(32, return_sequences=False),
    Dropout(0.2),
    Dense(1)  # Price prediction
])

model.compile(
    optimizer='adam', loss='mse')

model.fit(X_train, y_train,
    epochs=100, batch_size=16,
    callbacks=[EarlyStopping(
        patience=10)])
```

```python
# 9-Feature Input Vector
LSTM_FEATURES = [
    'price',          # Current
    'rainfall',       # Climate
    'wb_crude_oil',   # Global oil
    'wb_maize',       # Global maize
    'price_lag_1',    # 1 month ago
    'price_lag_2',    # 2 months ago
    'month_sin',      # Seasonal
    'month_cos',      # Seasonal
    'rain_lag_1'      # Rain 1mo ago
]

# MinMax Normalisation
scaler = MinMaxScaler()
train_sc = scaler.fit_transform(
    train[LSTM_FEATURES])

# Sliding Window: 6 months -> 1
X, y = create_sequences(
    train_sc, window=6)
```

LSTM: Training Curves & Forecast Results

Presenter: AYEBARE Moses *[21:00 – 22:00]*



Evaluation Metrics

Presenter: Apilli Barbra Ebal *[22:00 – 23:00]*

MAE

$$\frac{1}{n}\sum|y_i - \hat{y}_i|$$

Average error in UGX

Most interpretable

RMSE

$$\sqrt{\frac{1}{n}\sum(y_i - \hat{y}_i)^2}$$

Penalises large errors

Sensitive to outliers

MAPE

$$\frac{100\%}{n}\sum\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

Scale-independent

Cross-crop comparison

Evaluation Protocol

Strict chronological **80/20 split** – no shuffling → no data leakage → no future information in training

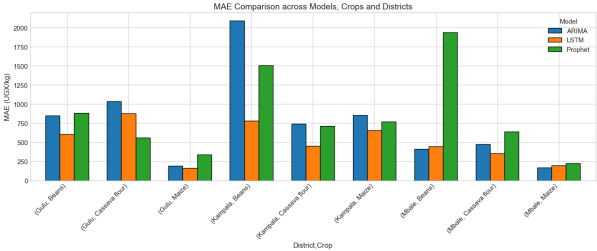
Aggregate Results: LSTM Dominates

Presenter: Apilli Barbra Ebal *[23:00 – 24:30]*

Model	MAE	RMSE	MAPE
ARIMA	757	882	25.1%
Prophet	841	970	29.7%
LSTM	**503**	**606**	**17.7%**

Why LSTM Wins

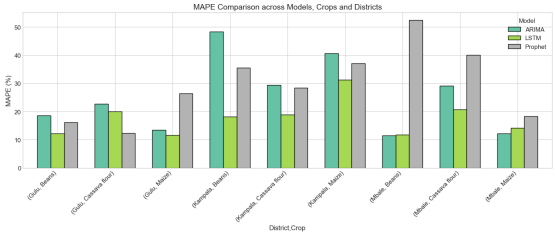
1. **Nonlinear** modelling capacity
2. **Multivariate** (9 features vs 1)
3. **Long-range memory** via gating



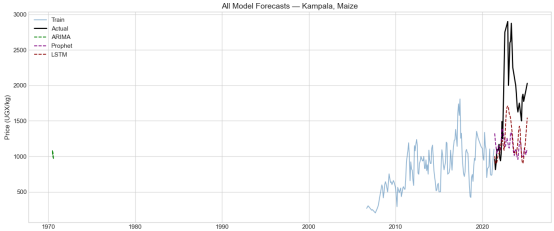
MAE comparison – LSTM is **33% lower** *than ARIMA*

MAPE Comparison & All-Models Overlay

Presenter: Apilli Barbra Ebal *[24:30 – 25:30]*



MAPE comparison – LSTM best in 7/9 pairs



All three models overlaid – LSTM (green) tracks actuals most closely

District-Level Performance Breakdown

Presenter: Apilli Barbra Ebal

District	Crop	ARIMA MAPE	Prophet MAPE	LSTM MAPE
Kampala	Maize	40.66%	37.13%	**31.33%**
Kampala	Beans	48.35%	35.59%	**18.15%**
Kampala	Cassava flour	29.40%	28.40%	**18.84%**
Gulu	Maize	13.49%	26.39%	**11.59%**
Gulu	Beans	18.60%	16.18%	**12.22%**
Gulu	Cassava flour	22.67%	**12.34%**	20.01%
Mbale	Maize	**12.25%**	18.37%	14.22%
Mbale	Beans	**11.52%**	52.50%	11.78%
Mbale	Cassava flour	29.08%	40.07%	**20.75%**

LSTM wins 7/9 • **ARIMA wins 2/9** (Mbale – simpler, stable market) • **Prophet wins 1/9** (strong annual seasonality)

Interactive Web Dashboard: Live Demo

Presenter: Apilli Barbra Ebal *[25:00 – 27:30]*

Dashboard Features:

1. **User Controls:** Select crop, district, horizon
2. **Real-time LSTM Training:** Model trains live
3. **Interactive Plotly Chart:**
   - Historical train data
   - Test actuals vs LSTM predictions
   - Future forecast trajectory
4. **Performance Metrics:** MAE, RMSE, MAPE
5. **Model Comparison Panel**: ARIMA vs Prophet vs LSTM

```
# Launch command
streamlit run app.py
```

```
# Dashboard core: app.py
import streamlit as st
import tensorflow as tf

st.title("Deep Market Forecaster")

# User selects parameters
district = st.selectbox(
    "District:",
    ['Kampala','Gulu','Mbale'])
crop = st.selectbox(
    "Crop:",
    ['Maize','Beans',
     'Cassava flour'])
horizon = st.radio(
    "Forecast Horizon:",
    ["4 Weeks", "8 Weeks"])

# Generate button triggers
# LSTM training + forecasting
if st.button("Generate Forecast"):
    model = train_lstm(data)
    forecast = predict(model)
    st.plotly_chart(fig)
```

Challenges Faced & Overcome

Presenter: Apilli Barbra Ebal *[27:30 – 29:00]*

1. Data Sparsity

Problem: Neural networks need 10,000+ rows; we had ∼200

Solution:

- **Dropout** (20%) – implicit ensemble
- **Early stopping** (patience=10)
- Compact architecture (64→32)

2. Geographic Mismatch

Problem: WFP ≠ CHIRPS ≠ World Bank

Solution: Manual mapping dicts + MS resampling

3. Multi-step Forecasting

Problem: Error compounds recursively

Solution: Recursive strategy with lag updates

Known Limitations

- Single train/test split (vs time-series CV)
- No confidence intervals
- ARIMA used univariate only
- Cannot predict black swan events

Conclusions & Future Work

Presenter: Apilli Barbra Ebal *[29:00 – 30:00]*

What We Achieved

1. Built **end-to-end ML pipeline**: data → features → models → evaluation → deployment
2. Merged **3 heterogeneous datasets** (WFP + CHIRPS + World Bank)
3. Trained and compared **3 fundamentally different models**
4. LSTM achieved **17.65% average MAPE** – competitive with published benchmarks
5. Deployed to **interactive Streamlit dashboard** with model comparison panel

Future Work

- Transformer architectures
- Model ensembling (learned weights)
- Additional covariates (conflict, mobility)
- Probabilistic forecasts (intervals)
- Cloud deployment (AWS/GCP)
- SMS/USSD for farmer access

Thank You

Questions & Discussion

Repository: `https://github.com/Jamugisa-Peter-Paul/Price-forcasting`
Dashboard: `streamlit run app.py`

Jamugisa Peter Paul • ILEGO Cynthia Cecilia • AYEBARE Moses • Apilli Barbra Ebal